

8x8 Multiplier Implementation (4x4 core, shift-and-add datapath) Week 2

이번 주차 과제에서는 **4x4 곱셈기 하나만으로 8x8 곱셈을 수행하는 회로**를 Verilog로 구현했다. 큰 곱셈기를 통째로 만드는 대신, 8bit 피연산자를 4bit 단위(nibble)로 쪼개서 부분곱을 여러 번 계산하고, 자리수(shift)를 맞춰서 누적(add)하는 **shift-and-add** 방식을 썼다. 이 글에서는 (1) 전체 데이터패스가 왜 이렇게 생겼는지, (2) 각 모듈이 데이터패스에서 어떤 역할을 하는지, (3) FSM 이 이 하드웨어를 어떻게 4 cycle 동안 재사용하는지를 순서대로 정리한다.

(설계도.jpg — Controller(FSM) + Stage 1~4 데이터패스)

왜 4x4 곱셈기 하나로 8x8을 계산할 수 있는가?

8bit 값 a , b 를 각각 상위 4bit(H)와 하위 4bit(L)로 나누면 다음처럼 쓸 수 있다.

```
1  a = 16*aH + aL      (aH = a[7:4], aL = a[3:0])
2  b = 16*bH + bL      (bH = b[7:4], bL = b[3:0])
```

이 둘을 곱해서 전개하면 부분곱 4개의 합이 된다.

```
1  a * b = aL*bL        (자리수 이동 0)
2      + aH*bL * 16      (자리수 이동 4bit, <<4)
3      + aL*bH * 16      (자리수 이동 4bit, <<4)
4      + aH*bH * 256     (자리수 이동 8bit, <<8)
```

→ 각 부분곱은 전부 4bit × 4bit 곱셈이다. 즉 4x4 곱셈기 하나만 있으면 되고, 대신 그것을 4번 돌리면서 자리수만 맞춰 더하면 8x8 결과가 나온다. 이게 이 설계의 핵심 아이디어다. 10진수 두 자리 곱셈을 세로셈으로 푸는 것과 똑같은 원리이며, 여기서는 base가 10이 아니라 16(nibble)이라고 보면 된다.

부분곱 4개와 자리수 이동을 표로 정리하면 다음과 같다.

순번(State)	data_a	data_b	부분곱	shift_sel	이동량	가중치
CALC_0	aL	bL	aL·bL	0	0	×1
CALC_1	aH	bL	aH·bL	1	CHUNK(4)	×16
CALC_2	aL	bH	aL·bH	1	CHUNK(4)	×16
CALC_3	aH	bH	aH·bH	2	WIDTH(8)	×256

전체 데이터패스 개요

설계도를 보면 데이터가 왼쪽에서 오른쪽으로 **Stage 1 → Stage 2 → Stage 3 → Stage 4**를 통과한다. 위쪽의 **Controller(FSM)** 는 이 데이터패스에 직접 데이터를 흘려보내지 않고, 각 stage가 "이번 cycle에 무슨 일을 할지" 정해주는 **제어 신호(sel_a, sel_b, shift_sel, acc_en, clr, done)** 만 뿌린다.

한 cycle에 데이터가 흐르는 경로는 이렇다.

1. a, b (8bit) 가 **selector**로 들어가서, FSM이 준 sel_a / sel_b 에 따라 상위/하위 nibble 하나씩 골라 data_a, data_b (4bit)를 뽑는다.
2. data_a, data_b 가 mul_4bit로 들어가 곱해져서 mul_init (8bit 부분곱)이 나온다.
3. mul_init 이 **shifter**로 들어가 shift_sel 만큼 왼쪽으로 밀려 16bit 폭의 mul 로 자리수가 맞춰진다.
4. mul 이 **adder**로 들어가 acc_en 이 1일 때 기존 누적값에 더해진다. 이 누적 레지스터가 최종 result[15:0] 이다.

여기서 중요한 점은 **Stage 1~3은 전부 조합회로(combinatorial)이고**, 레지스터는 **Stage 4의 accumulator 하나뿐**이라는 것이다. 그래서 이 회로는 "진짜 4단 파이프라인"이 아니라, **같은 하드웨어를 매 cycle 재사용하는 multi-cycle(반복형) 구조다**. 한 cycle에 부분곱 하나를 만들어 누적하고, 이것 4번 반복한다.

→ 조합 경로(selector→mul→shifter)로 부분곱을 만들고, 유일한 순차 소자인 accumulator에 매 cycle 더해 넣는다.

각 모듈별 상세 설명

※ Stage 1 — selector (nibble 선택 MUX)

8bit 피연산자에서 이번 cycle에 쓸 4bit nibble을 고르는 2-to-1 MUX 두 개다. sel_a 가 0이면 하위 nibble a[3:0], 1이면 상위 nibble a[7:4] 를 내보낸다. b 도 sel_b 로 똑같이 처리한다.



```
1 assign data_a = sel_a ? a[WIDTH-1:CHUNK] : a[CHUNK-1:0]; // 1:aH, 0:aL
2 assign data_b = sel_b ? b[WIDTH-1:CHUNK] : b[CHUNK-1:0]; // 1:bH, 0:bL
```

- WIDTH=8, CHUNK=4 파라미터라서 a[7:4] (상위) / a[3:0] (하위)로 정확히 반씩 잘린다.
- sel_a, sel_b 가 독립적인 1bit 신호이기 때문에 (aL,bL), (aH,bL), (aL,bH), (aH,bH) 네 가지 조합을 모두 만들 수 있다. 이 네 조합이 곧 위에서 본 부분곱 4개다.

→ selector는 "이번에 어느 자리 곱셈을 할지"를 nibble 단위로 골라주는 입력 게이트다.

※ Stage 2 — mul_4bit (4x4 조합 곱셈기)

이 설계에서 실제로 곱셈을 하는 유일한 연산기다. 4bit x 4bit를 곱해 최대 8bit 결과(0xF*0xF = 225)를 낸다.



```
1 assign mul_init = data_a * data_b; // [3:0] x [3:0] -> [7:0]
```

- 순수 조합회로라서 data_a, data_b 가 바뀌면 즉시 mul_init 이 갱신된다.
- 합성하면 이 * 하나가 FPGA의 곱셈 로직(또는 DSP)으로 매핑된다. 8x8을 한 방에 하지 않고 4x4만 두는 이유가 바로 연산기 크기를 작게 유지하기 위해서다.

※ Stage 3 — shifter (자리수 정렬)

부분곱 mul_init 을 최종 16bit 결과의 올바른 자리로 밀어 넣는 배럴 시프터다. FSM이 준 shift_sel 에 따라 이동량이 달라진다.



```

1  wire [2*WIDTH-1:0] mul_ext;
2  assign mul_ext = {{(2*WIDTH-2*CHUNK){1'b0}}, mul_init}; // 8bit -> 16bit zero-extend
3
4  always @(*) begin
5      case(shift_sel)
6          2'd0 : mul = mul_ext; // 이동 없음 (x1)
7          2'd1 : mul = mul_ext << CHUNK; // 4bit 왼쪽 이동 (x16)
8          2'd2 : mul = mul_ext << WIDTH; // 8bit 왼쪽 이동 (x256)
9          default : mul = 0;
10     endcase
11 end

```

- 먼저 mul_init (8bit)을 16bit로 **zero-extend**해서 shift 중에 상위 비트가 잘리지 않게 한다.
- shift_sel 이 0/1/2일 때 각각 <<0 / <<4 / <<8 이 되어 앞의 전개식에서 본 x1 / x16 / x256 가중치를 하드웨어로 구현한다.
- CALC_1과 CALC_2는 둘 다 shift_sel=1 (x16)이다. aH·bL 와 aL·bH 가 같은 자리(16의 자리)에 오기 때문이다.

→ shifter는 "이 부분곱이 몇 번째 자리인지"를 곱셈이 아니라 비트 이동으로 반영한다.

※ Stage 4 — adder (누적 레지스터, accumulator)

시프트된 부분곱 mul 을 매 cycle 더해 나가는 유일한 순차 회로다. 곱셈 결과가 여기에 쌓여서 최종 result[15:0] 이 된다.



```

1  always @(posedge clk or negedge rst_n) begin
2      if (!rst_n) result <= 0; // 비동기 리셋
3      else if (clr) result <= 0; // FSM이 IDLE에서 새 곱셈 전에 초기화
4      else if (acc_en) result <= result + mul; // 부분곱 누적
5      // 둘 다 0이면 값 유지(hold)
6  end

```

- clr 은 새 곱셈을 시작하기 전 누적값을 0으로 비운다. (직전 곱셈 결과가 남아 오염되는 것 방지)
- acc_en 이 1인 cycle에만 더한다. CALC_0~CALC_3 네 cycle 동안 acc_en=1 이라 부분곱 4개가 순서대로 누적된다.
- 우선순위가 rst_n > clr > acc_en 이라, 리셋/초기화가 누적보다 항상 앞선다.

→ adder는 데이터패스의 종착점이자 상태를 들고 있는 유일한 소자다. "4번 더하기"가 끝나면 이 레지스터가 곧 답이다.

Controller(FSM_mul) — 4 cycle 동안 하드웨어를 어떻게 재사용하는가

FSM은 데이터패스 위에서 "지금 몇 번째 부분곱 차례인지"를 세고, 그에 맞는 제어 신호를 뿌리는 두뇌다. IDLE → CALC_0 → CALC_1 → CALC_2 → CALC_3 → DONE → IDLE 로 한 바퀴 돈다.

상태 레지스터(cstate)와 다음 상태 논리, 출력 논리를 세 개의 always 블록으로 분리한 전형적인 **Moore FSM**이다.



```

1 // 1) 상태 레지스터
2 always @(posedge clk or negedge rst_n)
3     if (!rst_n) cstate <= IDLE;
4     else        cstate <= nstate;
5
6 // 2) 다음 상태 논리 (조합)
7 always @(*) begin
8     nstate = cstate;
9     case(cstate)
10        IDLE : nstate = start ? CALC_0 : IDLE; // start 눌리면 곱셈 시작
11        CALC_0 : nstate = CALC_1;
12        CALC_1 : nstate = CALC_2;
13        CALC_2 : nstate = CALC_3;
14        CALC_3 : nstate = DONE;
15        DONE : nstate = IDLE;
16        default: nstate = IDLE;
17    endcase
18 end

```

출력 논리는 상태마다 데이터패스 제어 신호를 세팅한다. 각 상태가 곧 위 표의 부분곱 한 줄과 1:1로 대응한다.

상태	sel_a	sel_b	shift_sel	acc_en	clr	done	하는 일
IDLE	0	0	0	0	1	0	accumulator 초기화, start 대기
CALC_0	0(aL)	0(bL)	0	1	0	0	aL·bL 누적
CALC_1	1(aH)	0(bL)	1	1	0	0	aH·bL <<4 누적
CALC_2	0(aL)	1(bH)	1	1	0	0	aL·bH <<4 누적
CALC_3	1(aH)	1(bH)	2	1	0	0	aH·bH <<8 누적
DONE	0	0	0	0	0	1	완료 신호

핵심은 **Stage 1~4의 하드웨어는 하나뿐인데, FSM이 매 cycle sel_a/sel_b/shift_sel 만 바뀌어서 같은 회로에 다른 부분곱을 돌려보낸다**는 점이다. 그래서 곱셈기도, 시프터도 한 벌만 있으면 되고, 대신 결과가 나오기까지 여러 cycle이 걸린다. (면적을 아끼는 대신 시간을 쓰는 전형적인 trade-off)

※ 데이터가 실제로 흐르는 타이밍

한 가지 주의할 점은, selector·mul·shifter는 조합회로라 상태에 맞는 부분곱을 그 **cycle 안에** 즉시 만들어내지만, adder는 클럭 엣지에서만 값을 잡는다는 것이다. 그래서 "CALC_0 상태 동안 만들어진 aL·bL"은 **CALC_0 → CALC_1로 넘어가는 posedge**에 accumulator로 들어간다. CALC_3의 부분곱은 CALC_3 → DONE 엣지에 마지막으로 더해지고, 그 다음 DONE 상태에서 done 이 1로 뜬다. 즉 done 이 뜨는 순간 result 에는 부분곱 4개가 전부 누적되어 있다.

검증 (testbench)

tb_top_mul_4bit.v 에서 클럭을 10ns 주기로 돌리고, run_test(a, b) task로 곱셈 한 건씩 수행했다. start 를 한 cycle 주고, done 이 뜰 때까지 기다린 뒤 result 를 Verilog 내장 곱셈(a*b)과 비교한다.



```

1  wait (done);
2  #1;
3  if (result == a_in * b_in)
4      $display("PASS : %0d x %0d = %0d", a_in, b_in, result);
5  else
6      $display("FAIL : ...");

```

테스트 벡터는 경계값 위주로 골랐다: 13 x 7 (일반값), 255 x 255 (최댓값, 16bit 꼭 참), 0 x 9 (0 처리). 손으로 shift-and-add 를 따라가 봐도 결과가 맞는다.

```

1  13 x 7   = 91      (aL·bL + aH·bL<<4 + aL·bH<<4 + aH·bH<<8)
2  255 x 255 = 65025  (15·15 + 15·15·16 + 15·15·16 + 15·15·256)
3  0 x 9    = 0

```

→ 세 벡터 모두 `result == a*b` 가 성립해 **PASS**했다. 부분곱 분해 → 시프트 → 누적이 8x8 곱셈과 수학적으로 동일함을 확인했다.

나의 질문

Q. CALC_1과 CALC_2는 왜 둘 다 `shift_sel=1` (같은 이동량)인가? 다른 부분곱인데 자리수가 같아도 되나?

A. $a_H \cdot b_L$ 과 $a_L \cdot b_H$ 는 서로 다른 곱셈이지만 **자릿값(가중치)**이 둘 다 16이라 같은 자리에 놓인다. 전개식 $a*b = a_L \cdot b_L + 16 \cdot a_H \cdot b_L + 16 \cdot a_L \cdot b_H + 256 \cdot a_H \cdot b_H$ 을 보면 가운데 두 항의 계수가 똑같이 16이다. 그래서 이동량(<<4)도 같고, accumulator가 알아서 둘을 같은 자리에서 합쳐준다. 자리(shift)는 "누구를 곱했나"가 아니라 "몇의 자리 곱인가"로 정해지기 때문이다.

결론 : 8bit를 nibble로 쪼개 4x4 곱셈기 하나로 부분곱 4개를 만들고(selector→mul_4bit), shifter로 x1/x16/x256 자리를 맞춘 뒤 accumulator에 4 cycle 누적하는 shift-and-add 데이터패스로 8x8 곱셈을 구현했으며, FSM이 매 cycle 제어신호만 바꿔 같은 하드웨어를 재사용한다. 경계값 테스트에서 `result == a*b`로 모두 PASS했다.